DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Annual Compounding Interest in Banking

CMSC 131 Lab 2

Prepared by: Rene Andre Bedonia Jocsing

Published: September 11, 2025

This laboratory assignment introduces fixed point integers and floating point numbers in the NASM assembly language, particularly in the context of banking. You will implement a program that simulates compounding interest over a period of time, using fixed point integer calculations. You will also convert from fixed point representation to floating point representation for output.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Store and manipulate monetary values in fixed point integer representation
- Convert from fixed point integer representation to floating point representation
- Perform percentage-based interest calculations in assembly
- Perform loops and persist data across iterations
- Implement control structures using jumps
- Perform modulo operations in assembly
- Follow proper assembly code structure and documentation practices

Task

Create a program that performs the following sequence:

1. Prompt the user to input an initial account balance (in centavos) as an integer.
2. Prompt the user to input an annual interest rate as a fixed point integer.
3. Prompt the user to input the number of years Y as an integer.
4. Simulate Y years of annual interest calculations.
5. After each year, display the current balance **in floating point representation**.

Specifications

- Initial balance: read as centavos in fixed point integer representation (i.e., 100000 centavos in fixed point is 1000.00 pesos in floating point).
- Annual interest rate: read as a percentage in fixed point integer representation (i.e., 375 in fixed point is 3.75% in percentage).
- Years: read as years in `int`.
- Use provided I/O functions: `print_string`, `print_int`, `read_int`.
- Assume all inputs are valid and within reasonable bounds.
- Do not use the FPU (floating point instructions) to convert from fixed point representation to floating point representation.

Notes

- Banks store your money in fixed point representation and only output the floating point representation at the end.
- It is possible to convert from fixed point integer representation to floating point representation with only non-FPU instructions.
- A Python script is provided to validate your results.

Implementation Details

Mathematical Formula

We use the following formula for calculating the annual compound interest in this assignment.

Balance_next = Balance_current + (Balance_current × Interest)/100

where all balances are in cents and Interest is scaled to avoid decimals.

Example: For 3.75% interest on 100000 cents (₱1000.00):

- Interest_cents = (100000 × 375)/10000 = 3750
- Balance_next = 100000 + 3750 = 103750 (₱1037.50)

Expected Output

```
Input initial balance in centavos: 100000
Input interest in fixed-point decimal: 375
Input number of years: 20
Year: 1 -> Balance: 1037.50
Year: 2 -> Balance: 1076.40
Year: 3 -> Balance: 1116.76
Year: 4 -> Balance: 1158.63
Year: 5 -> Balance: 1202.07
Year: 6 -> Balance: 1247.14
Year: 7 -> Balance: 1293.90
Year: 8 -> Balance: 1342.42
Year: 9 -> Balance: 1392.76
Year: 10 -> Balance: 1444.98
Year: 11 -> Balance: 1499.16
Year: 12 -> Balance: 1555.37
Year: 13 -> Balance: 1613.69
Year: 14 -> Balance: 1674.20
Year: 15 -> Balance: 1736.98
Year: 16 -> Balance: 1802.11
Year: 17 -> Balance: 1869.68
Year: 18 -> Balance: 1939.79
Year: 19 -> Balance: 2012.53
Year: 20 -> Balance: 2087.99
```

Think: Why do banks store your money in fixed point representation? Why not floating point?

Starter Code

No starter code template is provided for this exercise. *You can do it!*

Testing and Validation

Test your program with various balances and interest rates to ensure accuracy:

- Low rates (e.g., 1.0%)
- High rates (e.g., 12.5%)
- Long durations (e.g., 30 years)

A Python validation script is provided to verify your results against expected outputs.

Laboratory Defense

The instructor will be available during scheduled laboratory hours in the assigned room for defense or consultation. You will be catered to at a first-come first-served basis. It is mandatory to present your code, answer inquiries, and perform live programming if the instructor deems it necessary to further check your understanding.

If a laboratory defense for an activity fails to be conducted within **one month** of its assignment, the instructor will be forced to give a failing grade. Scoring will be done during the laboratory defense (no defense, no grade policy). Extensions may be granted due to extraordinary circumstances, but ultimately remains up to the judgment of the instructor.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course, harsh disciplinary action, and expulsion.

Rubric for Programming Exercises (50 pts)

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Program Correctness	Program executes correctly with no syntax or runtime errors, meets/exceeds specifications, and displays correct output	Program executes and outputs with minor errors, yet meets specifications	Program executes and outputs with major errors, yet somehow meets specifications	Program does not execute or does not meet specs
Logical Design	Program is logically well-designed with excellent structure and flow	Program has slight logic errors that do not significantly affect the results	Program has significant logic errors affecting functionality	Program logic is fundamentally incorrect
Code Mastery	Programmer demonstrates excellent mastery over the program's code	Programmer demonstrates adequate mastery over the program's code	Programmer demonstrates fair mastery over the program's code	Programmer demonstrates poor mastery over the program's code
Engineering Standards	Program is stylistically well designed from an engineering standpoint	Slight inappropriate design choices (i.e., poor variable names, improper indentation)	Severe inappropriate design choices (i.e., code repetition, redundancy)	Program is poorly written
Documentation*	Program is well-documented: comments exist for clarity, not redundancy	Missing one required comment or some redundant comments	Missing two or more required comments or many redundant comments	Most documentation missing or most documentation is redundant

***Remember: "Code tells you how, comments tell you why."** — Jeff Atwood, co-founder of Stack Overflow and Discourse

---

See the laboratory manual for submission requirements.